- Intro. Algorithms :

→ It is a step-by-step for solving a computational problem

→ But what is the diff. btw Alg & Program ?

→ Keep in mind ① First DESIGN ② IMPLEMENT

Priori Analysis	Posteriori testing
① DESIGN Algorithm	① IMPLEMENTATION PRogram
② DOMAIN Knowledge "Quem sabe sobre o problema"	② Progrommer
③ Any long (pseudo code)	③ prog. Long
④ Hw sw Independent	④ Hw Sw Dependent
⑤ Analyze time & space functions	⑤ testing watch time & bytes

→ Characteristics of Algorithms

- Input : 0 or +

UnKnown value

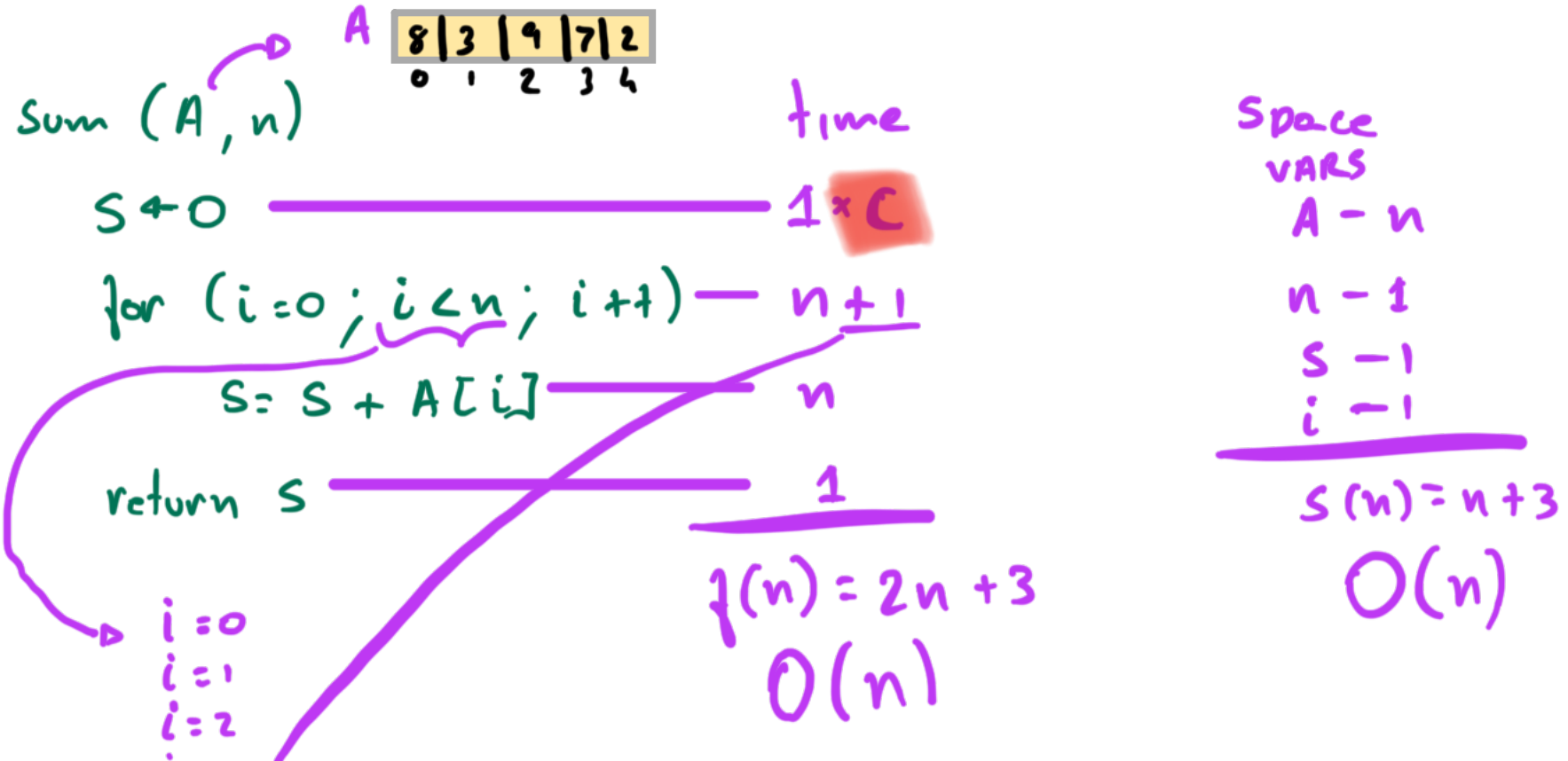
- output : 1 or + (at least 1 o/p)
- Definiteness : each statement must be well defined (output $\sqrt{-1}$ is NOT)
- Finiteness : each function must stop and return
- Effectiveness

→ How to analyze an algorithm

① time →
② Space → in terms of a function
③ Network consuption
④ power
⑤ CPU Register

→ Analizing time & Space

- Method freq. counting



i=3
i=4
i=5




```
Add (A, B, n)
  for (i=0; i<n; i++)
    for (j=0; j<n; j++)
      c[i,j] ← A[i,j] + B[i,j]
```

time

n+1

n x (n+1)

n x n

$f(n) = 2n^2 + 2n + 1$

$O(n^2)$

space

A - n^2

B - n^2

C - n^2

n - 1

i - 1

j - 1

$S(n) = 3n^2 + 3$

$O(n^2)$

```
mult(A, B, n)
  for (i=0; i<n; i++)
    for (j=0; j<n; j++)
      c[i,j] = 0
  for (k=0; k<n; k++)
    c[i,j] = c[i,j] + A[i,k] * B[k,j]
```

time

n+1

n (n+1)

n x n

n x n (n+1)

n x n x n

$f(n) = 2n^3 + 3n^2 + 2n + 1$

$O(n^3)$

space

A, B, C - n^2

VAR S - 1

$S(n) = 3n^2 + 4$

$O(n^2)$

others

① `for (i=0; i<n; i++)` → n+1

Something → n

$O(n)$

② `for (i=n; i>0; i--)` → n+1

Something → n

$O(n)$

③ `for (i=1; i<n; i=i+2)`

Something → $n/2$

$O(n)$

④ `for (i=1; i<n; i=i+20)`

Something → $n/20$

$O(n)$

⑤

```
for (i=0; i<n; i++)
    for (j=0; j<i; j++)
        something
```

i	j	# times something
0	0 ✓	0
1	0 ✓, 1 ✓	1
2	0 ✓, 1 ✓, 2 ✓	2
3	0 ✓, 1 ✓, 2 ✓, 3 ✓	3
n		n

$1+2+3+\cdots+n = \frac{n(n+1)}{2}$

$f(n) = O(n^2)$

⑥

```
p=0
for (i=1; p≤n; i++)
    p = p+i
```

i	p
1	0+1 = 1
2	1+2 = 3
3	1+2+3
4	1+2+3+4
⋮	⋮
K	1+2+3+4+⋯+K

Assuming $p > n$

$p = \frac{K(K+1)}{2}$

$\frac{K(K+1)}{2} > n$

Assuming $K = O(K^2)$

$K^2 > n$

$K = \sqrt{n}$

$\therefore O(\sqrt{n})$

⑦ for (i = 1 ; i < n ; i = i * 2)

something

Assuming $i \geq n$

$i = 2^k$

$2^k \geq n$

$2^k = n$

$k = \log_2 n$ ∴ $O(\log_2 n)$

i
1
$1 \times 2 = 2$
$2 \times 2 = 2^2$
$2^2 \times 2 = 2^3$
⋮
2^k

⑧ for (i = n ; i ≥ 1 ; i = i / 2)

something

Assuming $i < 1$

$\frac{n}{2^k} < 1$

$n = 2^k$

$k = \log_2 n$

$O(\log_2 n)$

i
n
$n/2$
$n/4$
$n/8$
⋮
$n/2^k$

⑨ p := 0

for (i = 1 ; i < n ; i = i x 2)

→ p ← p + 1

for (j = 1 ; j < p ; j++)

something

$\log n$

$\log p$

∴ $O(\log \log n)$

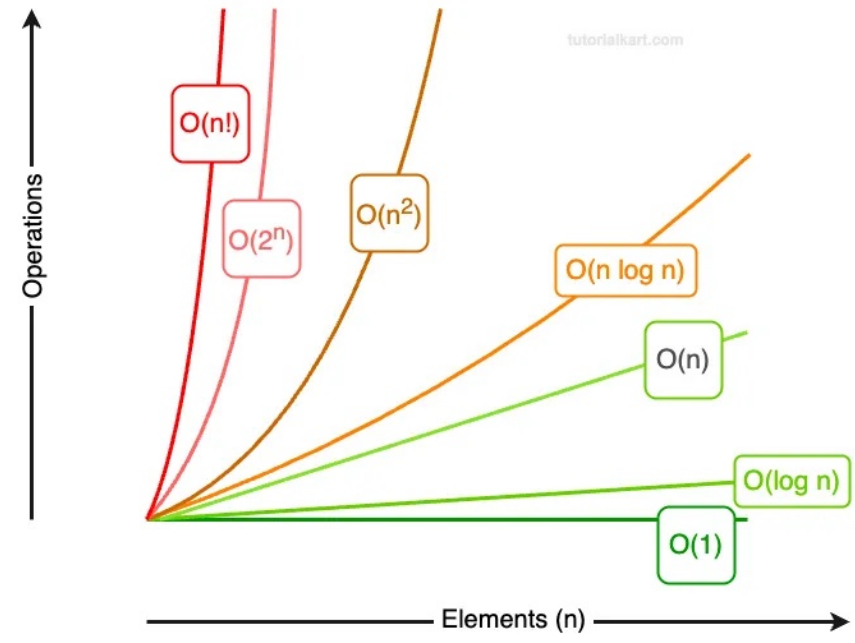
(10) for (i = 0; i < n; i++)
  for (j = 0; j < n; j = j * 2)
    something

n
n log n
n log n

2n log n + n

∴ O(n log n)

Big O Notation	Name	Example(s)
O(1)	Constant	# Odd or Even number, # Look-up table (on average)
O(log n)	Logarithmic	# Finding element on sorted array with **binary search**
O(n)	Linear	# Find max element in unsorted array, # Duplicate elements in array with Hash Map
O(n log n)	Linearithmic	# Sorting elements in array with **merge sort**
O(n^2)	Quadratic	# Duplicate elements in array **(naïve)**, # Sorting array with **bubble sort**
O(n^3)	Cubic	# 3 variables equation solver
O(2^n)	Exponential	# Find all subsets
O(n!)	Factorial	# Find all permutations of a given set/string



1 ≤ log n ≤ n ≤ n log n ≤ n^2 ≤ n^3 < 2^n < n!

log n	n	n^2	2^n
0	1	1	2
$\log_2^2 = 1$	2	4	4
2	4	16	16
3	8	64	256
3.1	9	81	512

→ Asymptotic Notation → comes from math!

O	big-oh	upper bound
Ω	big-omega	lower bound
Θ	theta	average bound

→ Big-oh :

the function $f(n) = O(g(n))$ iff $\exists$ positive constants c and n_0 such that

$$f(n) \leq c * g(n) \quad \forall\, n \geqslant n_0$$

$1 \leq \log n \leq n \leq n \log n \leq n^2 \leq n^3 < 2^n < n!$

lower bound

upper bound

Ex: $f(n) = 2n + 3$

$2n + 3 \leq 10\, n \qquad n \geqslant 1$

$f(n) \quad c \quad g(n)^* \quad n_0$

* We can use higher order function here! It will works.
But we intent to find the tightest function

→ Big-Omega ..

the function $f(n) = \Omega(g(n))$ iff $\exists$ positive constants c and n_0 such that

$$f(n) \geq c * g(n) \quad \forall n \geq n_0$$

Ex: $f(n) = 2n + 3$

$2n + 3 \geq 1 \times n$. $\quad n \geq 1$

$2n+3$ → $f(n)$; 1 → c; n → $g(n)$; $n \geq 1$ → n_0

→ theta

the function $f(n) = \Theta(g(n))$ iff $\exists$ positive constants c_1, c_2 and n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 * g(n) \quad \forall n \geq n_0$$

Ex: $f(n) = 2n + 3$

$1 \times n \leq 2n + 3 \leq 5 \times n$. $\quad n \geq 1$

c_1; $1 \times n$ → $g(n)$; $2n+3$ → $f(n)$; c_2; $5 \times n$ → $g(n)$; $n \geq 1$ → n_0

→ Propriedades

① If $f(n)$ is $O(g(n))$ then $\underline{a} * f(n)$ is $O(g(n))$

② Reflexive:

If $f(n)$ is given then $f(n)$ is $O(f(n))$

③ TRANSIVIVE

If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$

then $g(n)$ is $O(h(n))$

④ Symetric

If $f(n)$ is $\Theta(g(n))$ then $g(n)$ is $\Theta(f(n))$

⑤ transpose symetric

If $f(n) = O(g(n))$ then $g(n)$ is $\Omega(f(n))$

→ Best, worst, Average

- linear search

A	8	6	12	5	9	7	4	3	16	18
	0	1	2	3	4	5	6	7	8	9

Key = 7

Key = 20

Best case : Searching a element at first index

: $B(n) = O(1)$

worst case : Searching a key of least index

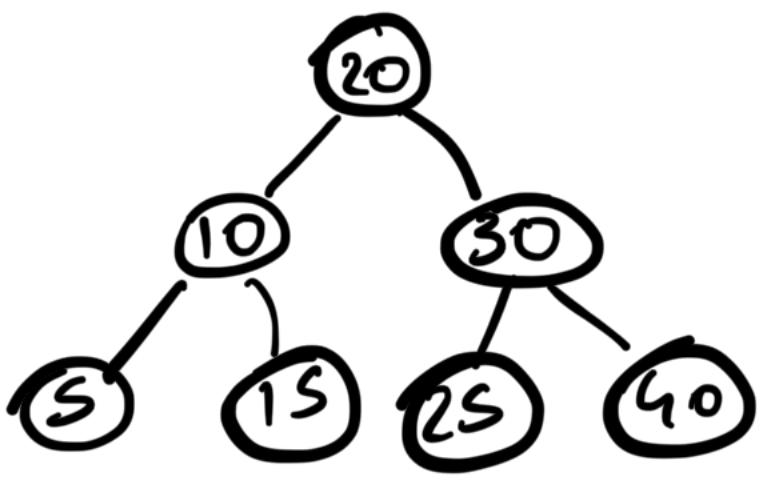
: $W(n) = O(n)$

Average case : $\frac{\text{all possible cases}}{\text{no cases}}$

: $Avg(n) = \frac{1+2+3+\dots n}{n} = \frac{\frac{n(n+1)}{2}}{n}$

$Avg(n) = \frac{n+1}{2} : O(n)$

- Binary search tree

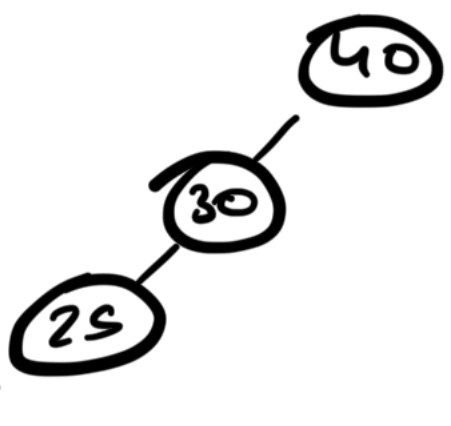


: Best case : element at root

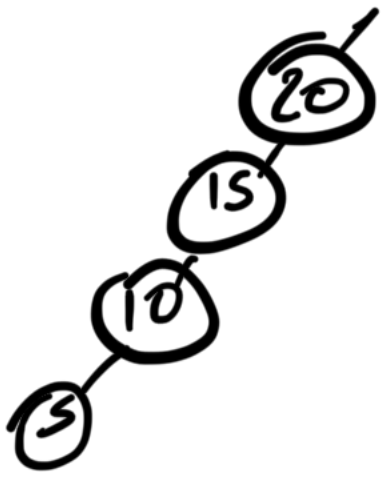
$B(n) = 1$

- Worst case : element at leaf.

height of the tree $O(h)$

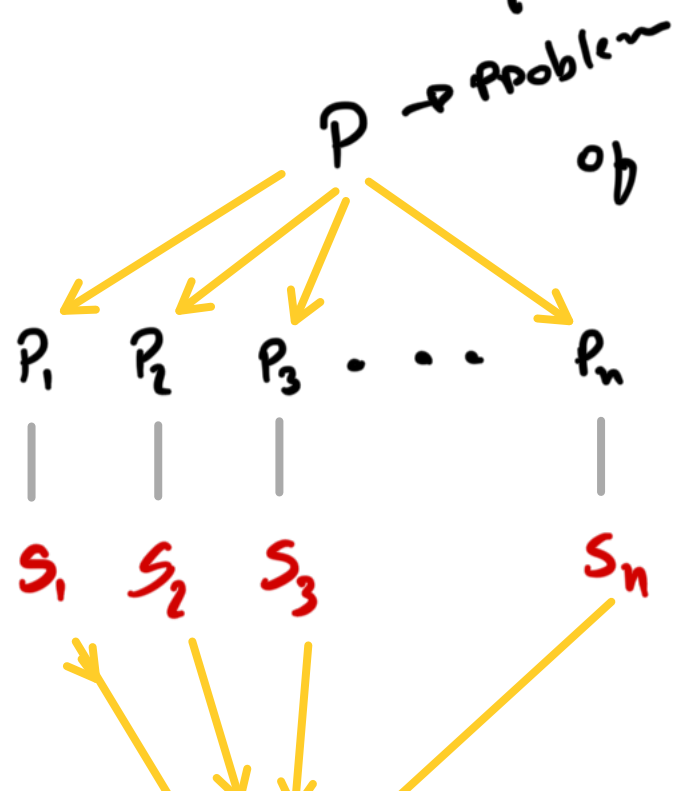


- Min $W(n) = O(\log n)$



- Max $W(n) = O(n)$

→ Divide & conquer



↝ it is not a sequence of tasks.

- they are subproblems



- the problem (eg. sorting) for small size is the same as the large one

DC (P)

```
if ( small (P) )
     solve (P)
else
   divide P into P1, P2 .... Pn
   Apply DC(P1), DC(P2),... DC(Pn)
   Combine (DC(P1), DC(P2)...)
```

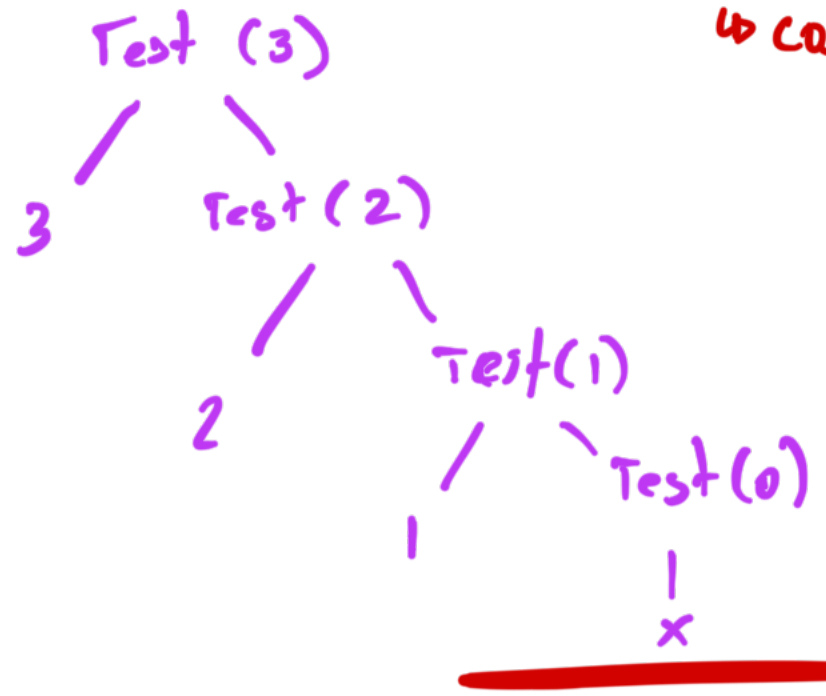
- Problems
  1. Binary Search
  2. Finding Max and Min
  3. Merg & Quick sorting
  4. Strassen's Matrix Mult.

-> Recurrence Relation

-> Method of freq.
  -> call x Instructions

①

```
Test ( n)
  if (n >0)
    print(n)
    Test (n-1)
```



In this example

```
Test (n)  ⇝ T(n)
  if (n >0)
    print(n) ⇝ 1
    Test (n-1) ⇝ T(n-1)
```

$$T(n) = T(n-1) + 1$$

$$T(n) = \begin{cases} 1 & , n = 0 \\ T(n-1) + 1 & , n > 0 \end{cases}$$

3 + 1 ⇝ in this example

n + 1 calls ⇝ in general

O(n) aproximation

$T(n) = T(n-1) + 1$

Substitute $T(n-1)$

$T(n-1) = T(n-1-1) + 1 = T(n-2) + 1$

$T(n-2) = T(n-2-1) + 1 = T(n-3) + 1$

$T(n) = [T(n-2) + 1] + 1$

$T(n) = T(n-2) + 2$

$T(n) = [T(n-3) + 1] + 2$

$T(n) = T(n-3) + 3$

⋮ continue for K times

$T(n) = T(n-K) + K$

Assuming $n - K = 0$.

$n = K$

$T(n) = T(n-n) + n$

$T(n) = 1 + n$

$O(n)$

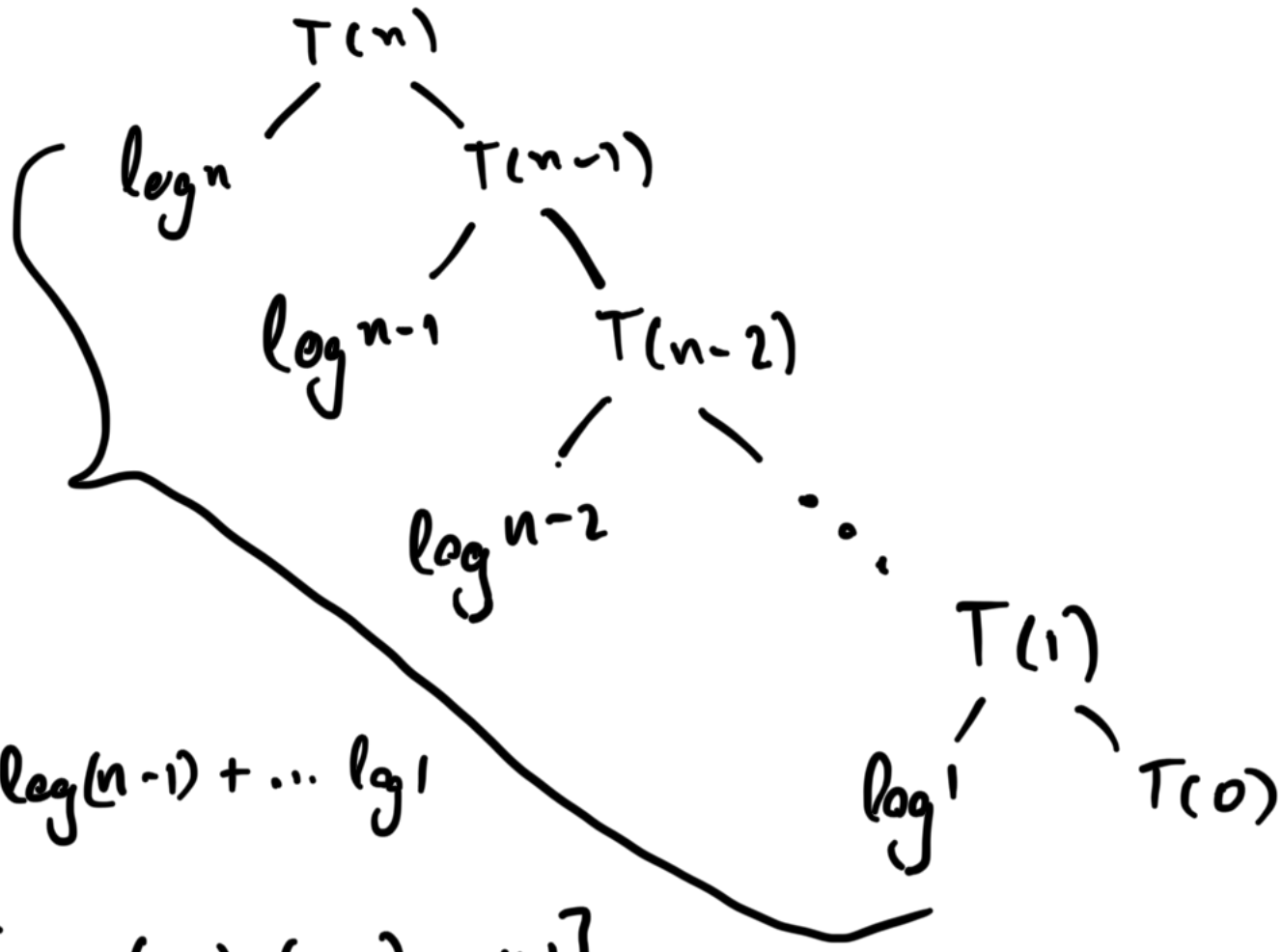
$$T(n) = \begin{cases} 1 & , n = 0 \\ T(n-1) + 1 & , n > 0 \end{cases}$$

②
```
Test (n)
  if (n>0)
    for (i=1; i<n; i=i*2)
      print (i)
```
⇝ log n

Teste (n-1) ⇝ T(n-1)

$T(n) = T(n-1) + \log n$

$$T(n) = \begin{cases} 1 & , n=0 \\ T(n-1) + \log n & , n>0 \end{cases}$$



$\log n + \log(n-1) + \dots \log 1$

$\log [n \times (n-1) \times (n-2) \dots \times 1]$

$\log n! \rightsquigarrow O(n \log n)$

Prove que $\log n!$ é $O(n \log n)$

$n! = 1 \times 2 \times 3 \dots \times n$

$\log n! = \log(1 \times 2 \times 3 \dots \times n)$

$\log n! = \log 1 + \log 2 + \log 3 + \dots + \log n$

com $\forall K \leq n$, $\log K \leq \log n$

$\log \underline{K} \leq \log n$

$\log \underline{n!} \leq \underbrace{\log n + \dots + \log n}_{n \text{ vezes}}$

logo

$\log n! \leq n \log n$

⇝ Some recurrence relations

$T(n) = T(n-1) + 1 \quad - O(n)$

$T(n) = T(n-1) + n - O(n^2)$

$T(n) = T(n-1) + \log n - O(n \log n)$

$T(n) = T(n-1) + n^2 - O(n^3)$

$T(n) = T(n-2) + 1 \rightsquigarrow n/2$

$O(n)$

$T(n) = 2T(n-1) + 1 \rightarrow O(2^n)$

$\therefore O(n \log_n)$